



6



22



2640 sp

نظري

كلية الهندسة المعلوماتية

السنة الثالثة

Using JavaScript

د. غاندي هسام



RB Informatics; 19/11/2025

لغات البرمجة

ما هو الـ DOM (Document Object Model):

يعد (DOM) جسراً أساسياً بين لغة JavaScript ومحتوى صفحات HTML. إن فهم كيفية عمل الـ DOM هو المفتاح لبناء تطبيقات ويب تفاعلية وديناميكية، حيث يسمح لنا بتجاوز حدود الصفحات الثابتة والانتقال إلى عالم المحتوى الحي الذي يستجيب للمستخدم.

لماذا نحتاج الـ DOM وما هو:

عند تشغيل JavaScript داخل المتصفح، يكون هناك ملف HTML معروض على الشاشة. JavaScript لوحدها لا تفهم HTML مباشرة، لذلك تحتاج إلى وسيط يسمح لها بالتعامل مع عناصر الصفحة. وهو DOM.

DOM هو واجهة برمجية (API) يوفرها المتصفح، تمثل صفحة الـ HTML بشكل كائنات (Objects) يمكن لـ JavaScript التعامل معها.

يعني:

كل عنصر HTML مثل (<div>, <p>,) يصبح كائن
كل خاصية (attribute) مثل: id, class, src تصبح خاصية للكائن

كيف يمثل الـ DOM الصفحة؟

الـ DOM يمثل الصفحة على شكل شجرة: (Tree Structure)

- الجذر هو document:

- تحته عناصر مثل:

- <html>

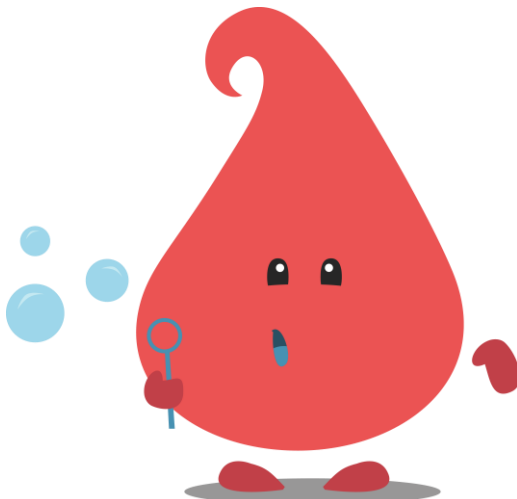
- <head>

- <body>

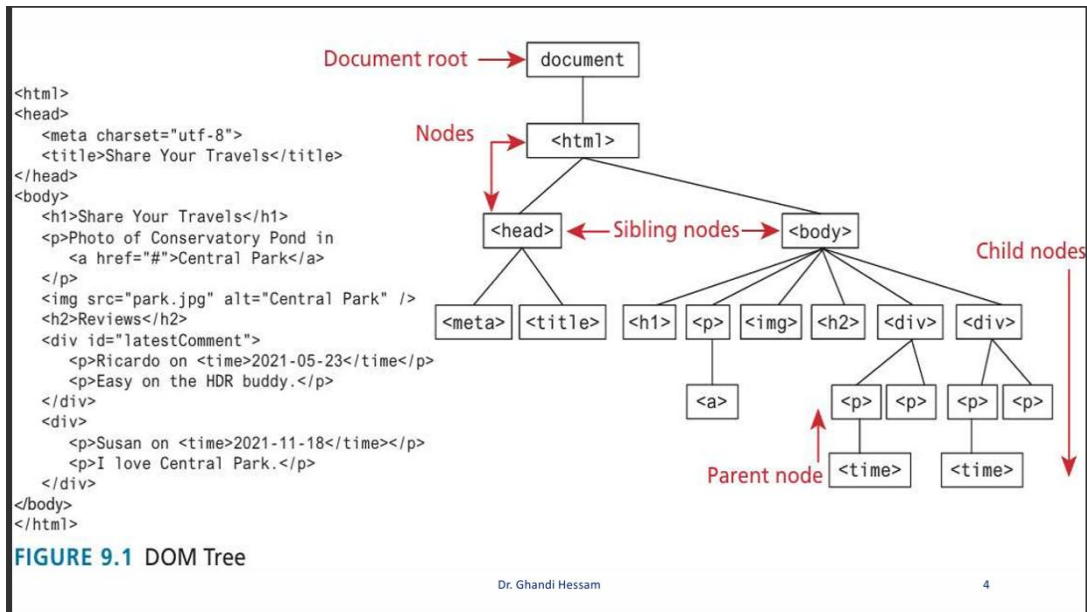
- <div>

- <p>

- <button>



❖ هذا يسمى هيكل هرمي (Hierarchical):



❖ ماذا يتيح لنا ال DOM ؟

من خلال DOM + JavaScript يمكننا:

- الوصول لأي عنصر في الصفحة، تغيير النص أو اللون أو الشكل، إضافة عناصر جديدة أو حذف عناصر
- قراءة وتعديل الخصائص (attributes) والتفاعل مع المستخدم (نقر، كتابة، تمرير...)

❖ العمل بشكل ديناميكي (Runtime)

ال DOM يسمح بالتعديل:

- أثناء تشغيل الصفحة
- استجابة لأحداث المستخدم مثل:
 - الضغط على زر
 - تحريك الماوس
 - إدخال نص

يعني الصفحة لا تبقى ثابتة، بل **تتغير لحظياً** حسب تفاعل المستخدم.

أي:

- DOM تمثيل HTML ككائنات يمكن التحكم بها
- هو الجسر بين JavaScript و HTML
- يوفر طريقة موحدة ومنظمة للوصول لعناصر الصفحة
- يسمح بإنشاء صفحات تفاعلية وديناميكية

البنات الأساسية للـ DOM: العُقد (Nodes) وقوائم العُقد (NodeLists)

إن فهم مفهوم "العُقدة (Node)" هو الخطوة الأولى لإتقان التعامل مع بنية الـ DOM فكل شيء في مستند HTML، بدءاً من الوسوم الرئيسية وانتهاءً بالنصوص البسيطة، يُمثَّل كعقدة في هذه الشجرة الهرمية.

ببساطة، العقدة هي كل عنصر داخل مستند HTML في سياق الـ DOM

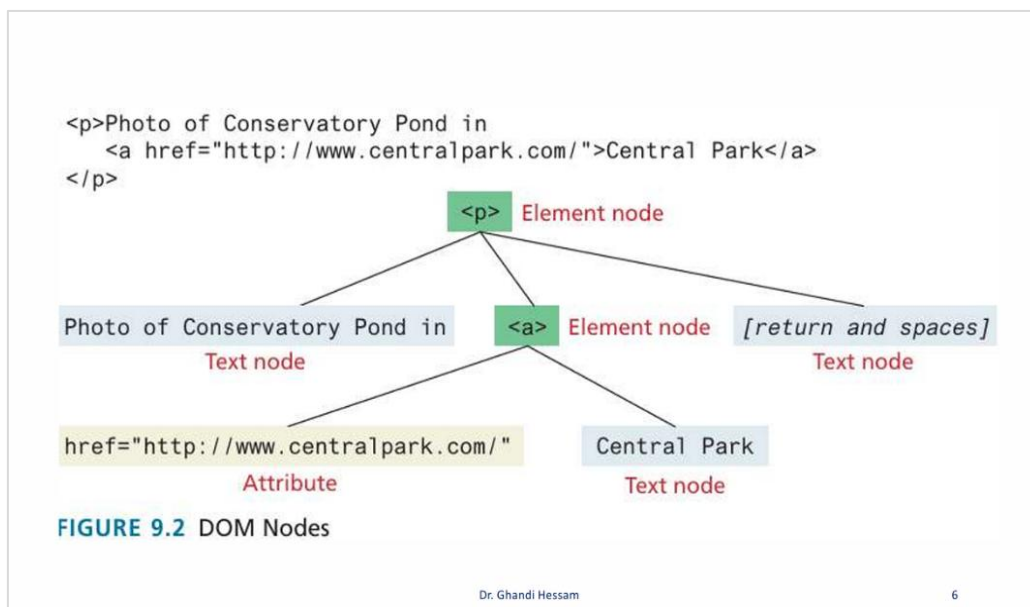
يمكننا التمييز بين نوعين رئيسيين من العقد:

- عُقد العناصر (Element Nodes): تمثل وسوم HTML مثل `<p>` أو `<a>` أو `<div>`.
- عُقد النصوص (Text Nodes): تمثل المحتوى النصي الموجود داخل عقد العناصر مثل النص داخل `<p>` أو `<span>`

مثال:

- `<button>Click me</button>`
- `<button>` → Element Node
- Click me → Text Node

ملاحظة: في الإصدارات القديمة من: DOM الصفات مثل `id` و `class` كانت تعتبر Attribute Node كانت "Attribute nodes" جزءاً من المواصفات الأصلية للـ DOM ، ولكن تم إهمالها رسمياً منذ إصدار DOM 4 في عام 2015، وأصبحت السمات الآن خصائص مباشرة لعقد العناصر.



تشارك جميع العقد في مجموعة من الخصائص والأساليب التي تسمح لنا باسترداد المعلومات ومعالجتها.

- nodeName → اسم العقدة
- nodeType → نوع العقدة (عنصر، نص...)
- parentNode → العقدة الأب
- childNodes → العقد الأبناء
- textContent → محتوى النص

↪ العلاقات بين العقد

Property	Description
childNodes	A <code>NodeList</code> of child nodes for this node
firstChild	First child node of this node
lastChild	Last child of this node
nextSibling	Next sibling node for this node
nodeName	Name of the node
nodeType	Type of the node
nodeValue	Value of the node
parentNode	Parent node for this node
previousSibling	Previous sibling node for this node
textContent	Represents the text content (stripped of any tags) of the node

TABLE 9.1 Some Essential Node Object Properties

Dr. Ghandi Hssam

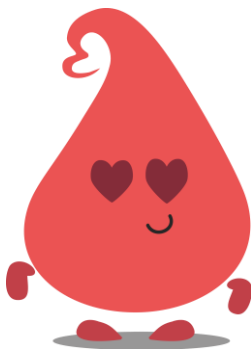
7

↪ نقطة الانطلاق: كائن المستند (Document Object)

يُعدّ كائن `document` المدخل الأساسي لجميع عمليات التفاعل مع نموذج كائن المستند (DOM) إذ يمثل هذا الكائن الجذر الذي يحتوي على كامل مستند HTML ، ومن خلاله يمكن الوصول إلى أي عنصر أو محتوى داخل الصفحة.

وكغيره من كائنات JavaScript ، يمكن التعامل مع خصائص كائن `document` باستخدام:

- باستخدام النقطة (Dot Notation)
- أو باستخدام الأقواس المربعة (Square Bracket Notation)



// استرداد عنوان URL للصفحة الحالية

```
let a = document.URL;
```

// استرداد ترميز الصفحة، مثل ISO-8859-1

```
let b = document["inputEncoding"];
```

ولأغراض تنظيمية، يمكن تصنيف الأساليب التي يوفرها كائن document إلى ثلاث فئات رئيسية:

1. أساليب التحديد (Selection Methods)

وتستخدم للعثور على العناصر داخل الصفحة.

2. أساليب معالجة البنية (Family Manipulation Methods)

وتستخدم لإنشاء العناصر أو حذفها أو تعديل بنيتها.

3. أساليب الأحداث (Event Methods)

وتستخدم للاستجابة لتفاعلات المستخدم مثل النقر والتمرير والكتابة.

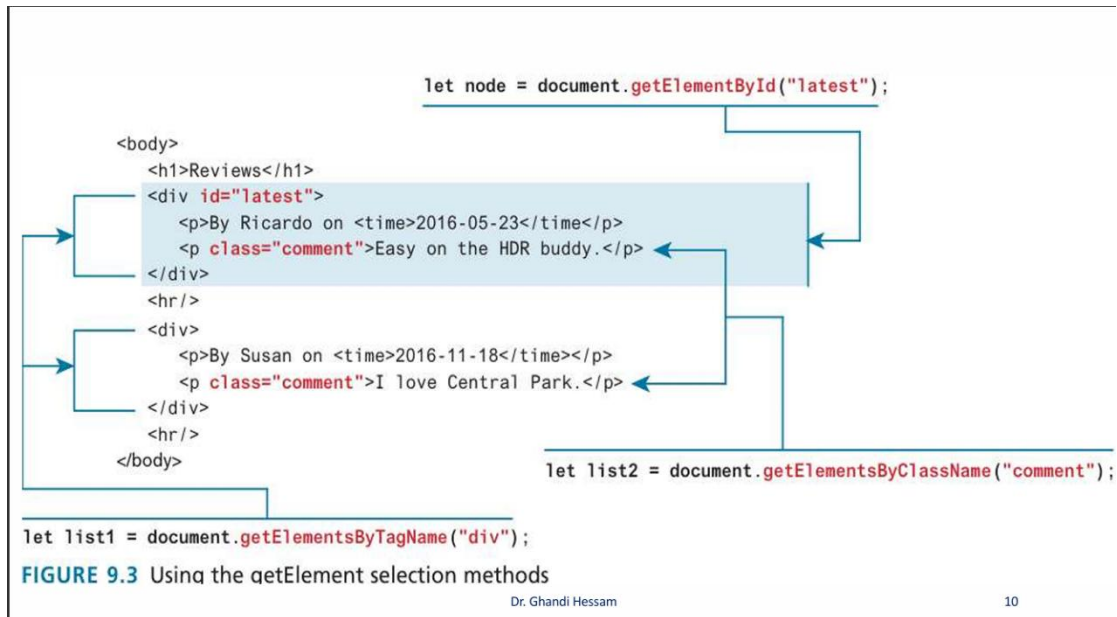
✎ العثور على العناصر: أساليب التحديد (Selection Methods)

تعد عملية تحديد العناصر الخطوة الأولى والأساسية لمعالجة أي جزء من صفحة الويب.

يوفر نموذج DOM مجموعة قوية من الأساليب التي تمكّن المطور من تحديد العناصر اعتمادًا على:

- المعرّف (id)
- اسم الفئة (class)
- اسم الوسم (tag)
- أو باستخدام محددات CSS المتقدمة

الوصف (Description)	الأسلوب (Method)
يُرجع عقدة العنصر الوحيدة التي يتطابق مُعرّفها id مع المُعرّف المُمرّر.	<code>getElementById("id")</code>
يُرجع <code>NodeList</code> من العناصر التي يتطابق اسم الفئة (class) الخاص بها مع الاسم المُمرّر.	<code>getElementsByClassName("name")</code>
يُرجع <code>NodeList</code> من العناصر التي يتطابق اسم الوسم (tag) الخاص بها مع الاسم المُمرّر.	<code>getElementsByTagName("name")</code>
يُرجع أول عقدة عنصر تتطابق مع مُحدّد CSS المُمرّر.	<code>querySelector("selector")</code>
يُرجع <code>NodeList</code> من العناصر التي تتطابق مع مُحدّد CSS المُمرّر.	<code>querySelectorAll("selector")</code>



`document.getElementById ("latest")`

يُرجع عنصر `<div>` الوحيد الذي يحمل هذا المعرّف.

`getElementsByClassName ("comment")`

يُرجع قائمة تحتوي على عنصري `<p>` يشتركان في نفس الفئة.

`getElementsByTagName ("div")`

يُرجع جميع عناصر `<div>` الموجودة داخل عنصر `<body>` في شكل `NodeList`.

⚡ عقدة العنصر (Element Node Object)

- الكائن الذي ترجعه الأساليب مثل `getElementById()` و `querySelector()` هو كائن من نوع

`Node Element` يمثل هذا الكائن عنصرًا واحدًا من عناصر HTML داخل الصفحة.

- يمكن لعقدة العنصر أن تحتوي بداخلها عناصر أخرى، مما يؤدي إلى بنية متداخلة تشبه الشجرة.

كما تمتلك جميع عناصر HTML مجموعة من الخصائص العامة التي تتيح قراءة معلومات العنصر أو تعديلها.

- إضافةً إلى ذلك، تحتوي بعض وسوم HTML المحددة (مثل `<input>` و `<img>` و `<a>`) على خصائص

إضافية خاصة بها، تختلف حسب نوع الوسم.

`<a>` له `href` ○

`<img>` له `src` ○

`<input>` له `value` ○





بعد تحديد عنصر معين من الصفحة، يمكن الوصول إلى خصائصه للتعامل مع محتواه أو مظهره أو سلوكه. الأساليب مثل `getElementById()` و `querySelector()` ترجع عنصراً من نوع `Element Node` يحتوي على العديد من الخصائص المفيدة.

↪ الخصائص الأساسية لعقدة العنصر

الخاصية	الوصف
<code>classList</code>	قائمة للقراءة فقط تحتوي على جميع فئات CSS الخاصة بالعنصر
<code>className</code>	القيمة النصية الحالية لسمـة <code>class</code>
<code>id</code>	قيمة المعرّف <code>id</code> للعنصر
<code>innerHTML</code>	محتوى HTML الداخلي للعنصر بما في ذلك الوسوم المتداخلة
<code>style</code>	كائن يمثل أنماط CSS المطبقة مباشرة على العنصر
<code>tagName</code>	اسم وسم HTML الخاص بالعنصر

ملاحظة: سمة النمط (style) للعنصر، تُرجع كائن `CSSStyleDeclaration`.

بالإضافة إلى هذه الخصائص العامة، تحتوي بعض وسوم HTML المحددة على خصائص إضافية خاصة بها، كما هو موضح في الجدول التالي:

الخاصية	الوصف	الوسوم
<code>href</code>	تحدد عنوان URL للرابط	<code><a></code>
<code>name</code>	تُستخدم لتعريف الوسم، وغالباً في النماذج	<code><a></code> , <code><input></code> , <code><textarea></code> , <code><form></code>
<code>src</code>	تحدد مصدر ملف خارجي مثل صورة أو سكريبت	<code></code> , <code><input></code> , <code><iframe></code> , <code><script></code>
<code>value</code>	تمكّن من قراءة أو تعديل قيمة إدخال المستخدم	<code><input></code> , <code><textarea></code> , <code><submit></code>



هذه الأمثلة توضّح كيف نصل إلى خصائص العناصر بعد تحديدها من الـ DOM باستخدام JavaScript

```
<p id="here">hello <span>there</span></p>
<ul>
  <li>France</li>
  <li>Spain</li>
  <li>Thailand</li>
</ul>
<div id="main">
  <a href="somewhere.html"></a>
</div>

<script>
  const node = document.getElementById("here");
  // outputs: hello <span>there</span>
  console.log(node.innerHTML);
  // outputs: hello there
  console.log(node.textContent);
```

Dr. Ghandi Hssam

14

```
const items = document.getElementsByTagName("li");
for (let i=0; i<items.length; i++) {
  // outputs: France, then Spain, then Thailand
  console.log(items[i].textContent);
}

const link = document.querySelector("#main a");
// outputs: somewhere.html
console.log(link.href);

const img = document.querySelector("#main img");
// outputs: whatever.gif
console.log(img.src);
// outputs: thumb
console.log(img.className);
</script>
```

LISTING 9.1 Accessing elements and their properties

Dr. Ghandi Hssam

15

التعامل مع محتوى عنصر نصّي

```
const node = document.getElementById("here");
```

هذا السطر يحدد عنصر HTML يحمل المعرّف here (الفقرة <p>).

```
console.log(node.innerHTML);
```

تطبع المحتوى الداخلي كاملاً للعنصر.

يشمل النص + أي وسوم HTML موجودة بداخله.

```
console.log(node.textContent);
```

تطبع النص فقط دون أي وسوم HTML.

🔗 الوصول إلى رابط (Link)

```
const link = document.querySelector("#main a");
```

- يحدد أول عنصر `<a>` موجود داخل العنصر الذي يحمل المعرف `main`.
- `console.log(link.href);`
- تطبع قيمة الخاصية `href` الخاصة بالرابط.
- أي عنوان الصفحة أو الملف الذي يشير إليه الرابط.

🔗 الوصول إلى صورة (Image)

```
const img = document.querySelector("#main img");
```



- يحدد أول عنصر `<img>` داخل العنصر `main`.
- `console.log(img.src);`
- تطبع مسار الصورة المرتبط بالعنصر.
- أي الملف الذي يتم تحميله وعرضه كصورة.

🔗 تعديل المحتوى innerHTML: مقابل textContent

تستخدم الخاصية `innerHTML` لإدراج أو تعديل محتوى HTML كامل داخل عنصر معين، مما يسمح بإنشاء عناصر جديدة مثل الصور والروابط. ومع ذلك، فإن استخدامها داخل الحلقات التكرارية أو مع مدخلات المستخدم قد يؤدي إلى مشكلات في الأداء والأمان.

• استخدام innerHTML

الخاصية `innerHTML` تسمح لك بإدخال أو تعديل HTML كامل داخل عنصر معين.

```
const div = document.querySelector("#main");
div.innerHTML = '<a href="#"></a>';
```

هنا

- يتم حذف أي محتوى سابق داخل `<div id="main">`
- ثم يتم وضع رابط `<a>` بداخله صورة `<img>`

يعني: أنت لا تضيف نصاً فقط، بل تغيّر بنية HTML نفسها.

مثال شائع: إنشاء قائمة `<ul>` ديناميكياً

هناك طريقتان شائعتان باستخدام `innerHTML`:

1. بناء النص كاملاً ثم إضافته مرة واحدة

- تجمع كل عناصر داخل نص واحد
- ثم تسنده إلى innerHTML مرة واحدة

هذه الطريقة أسرع نسبياً.

```
// the <ul> already exists somewhere
const list = document.querySelector("#main ul");
// first approach, construct string with content first
let items = '';
for (let i=0; i<5; i++) {
    items += '<li>Item ' + i + '</li>';
}
// then assign innerHTML to content
list.innerHTML = items;
```

2. الإضافة التدريجية داخل حلقة

- في كل دورة من الحلقة:
- ul.innerHTML += "عنصر";

هذه الطريقة أبطأ جداً، لأن:

- المتصفح يعيد تحليل الـ HTML
- ويعيد حساب تخطيط الصفحة (reflow)
- في كل مرة تتكرر فيها الحلقة

```
// second approach, add content to innerHTML incrementally
list.innerHTML = '';
for (let i=0; i<5; i++) {
    list.innerHTML += '<li>Item ' + i + '</li>';
}
```

من ناحية الأداء، تؤدي الإضافات المتكررة إلى innerHTML إلى إعادة تحليل وبناء شجرة DOM في كل مرة، مما يجعل هذه الطريقة غير فعالة. أما من الناحية الأمنية، فإن إدراج مدخلات المستخدم مباشرة باستخدام innerHTML يعرض التطبيق لهجمات البرمجة النصية عبر المواقع (XSS)، حيث يمكن تنفيذ تعليمات JavaScript ضارة داخل المتصفح. لذلك، يُوصى باستخدام الخاصية textContent عند الحاجة إلى تعديل النص فقط، لأنها أكثر أماناً وأسرع كونها لا تقوم بتحليل HTML أما في الحالات التي تتطلب إنشاء بنية HTML معقدة، فإن أفضل الممارسات تتمثل في استخدام أساليب معالجة الـ DOM مباشرة لإنشاء العناصر وإدارتها بشكل آمن واحترافي.

٤ البناء الديناميكي والتفاعل مع المستخدم في الـ DOM (DOM Manipulation Methods)

بدل الاعتماد على innerHTML لتعديل محتوى الصفحة، يوفر الـ DOM طريقة أكثر أماناً واحترافية للتعامل مع عناصر الصفحة، وذلك من خلال ما يُعرف بأساليب معالجة الـ DOM. هذا الأسلوب يسمح لنا بإنشاء عناصر جديدة، حذف عناصر موجودة، أو تعديل بنية الصفحة بشكل كامل عبر JavaScript، مع تجنب المشاكل الأمنية والأدائية المرتبطة بتحليل HTML كمنص.

رغم أن الـ DOM يوفر خصائص للتنقل بين العقد مثل firstChild و nextSibling، إلا أن استخدامها قد يكون مربكاً أحياناً بسبب وجود عقد نصية غير مرئية (مثل المسافات والأسطر الجديدة). لذلك، يُفضّل غالباً الاعتماد على أساليب التحديد مثل getElementById و querySelector للوصول إلى العناصر بشكل أدق وأكثر موثوقية.

يوضح الجدول التالي أهم أساليب معالجة الـ DOM:

الأسلوب (Method)	الوصف (Description)
<code>appendChild</code>	يضيف عقدة ابن جديدة إلى نهاية العقدة الحالية.
<code>createAttribute</code>	يُنشئ عقدة سمة جديدة.
<code>createElement</code>	يُنشئ عقدة عنصر HTML جديدة (مثل <p> أو <div>).
<code>createTextNode</code>	يُنشئ عقدة نصية جديدة.
<code>insertAdjacentElement</code>	يُدخّل عقدة جديدة في أحد المواضع الأربعة بالنسبة للعقدة الحالية.
<code>insertAdjacentText</code>	يُدخّل عقدة نصية جديدة في أحد المواضع الأربعة بالنسبة للعقدة الحالية.
<code>removeChild</code>	يزيل عقدة ابن من العقدة الحالية.
<code>removeChild</code>	يزيل عقدة ابن من العقدة الحالية.
<code>replaceChild</code>	يستبدل عقدة ابن بعقدة أخرى.
<code>insertBefore</code>	يُدخّل عقدة جديدة قبل عقدة مرجعية في العقدة الحالية.

دعونا نرى مثالاً عملياً يوضح كيفية إضافة عنصر فقرة <p> جديد إلى <div> موجود بالفعل، خطوة بخطوة:

```
<div id="first">
  <h1>DOM Example</h1>
  <p>Existing element</p>
</div>

<script>
  // begin by creating two new nodes
  const text = document.createTextNode("this is dynamic");
  const p = document.createElement("p");

  // add the text node to the <p> element node
  p.appendChild(text);

  // now add the new <p> element to the <div>
  const first = document.getElementById("first");
  first.appendChild(p);
</script>
```

21

1 Create a new text node

"this is dynamic"

```
const text = document.createTextNode("this is dynamic");
```

2 Create a new empty <p> element

<p></p>

```
const p = document.createElement("p");
```

3 Add the text node to new <p> element

<p> "this is dynamic" </p>

```
p.appendChild(text);
```

4 Add the <p> element to the <div>

```
const first = document.getElementById("first");
first.appendChild(p);
```

```
<div id="first">
  <h1>DOM Example</h1>
  <p>Existing element</p>
  <p>this is dynamic</p>
</div>
```

```
<div>
  <h1> "DOM Example" </h1>
  <p> "Existing element" </p>
  <p> "this is dynamic" </p>
</div>
```

22

1. إنشاء عقدة نصية: نُنشئ المحتوى النصي للفقرة.

2. إنشاء عنصر الفقرة: نُنشئ عنصر <p> فارغاً.

3. إضافة النص إلى الفقرة: نضع العقدة النصية داخل عنصر الفقرة.

4. إضافة الفقرة المكتملة إلى المستند: نحدد العنصر الأب ونضيف إليه الفقرة الجديدة.

بعد أن تعلمنا كيفية بناء وتعديل الصفحة ديناميكياً، حان الوقت لتتعلم كيفية جعلها تتفاعل مع المستخدم عبر نظام الأحداث.

Event methods

بعد إتقان إنشاء وتعديل العناصر ديناميكياً، ننتقل إلى جانب مهم جداً في تطبيقات الويب، وهو التفاعل مع المستخدم. هنا يأتي دور نظام الأحداث (Events)، وهو الآلية التي تسمح لصفحات الويب بالاستجابة لأفعال المستخدم مثل النقر، تحريك الفأرة، أو إدخال النص من لوحة المفاتيح.

1. عند التعامل مع الأحداث، يتم أولاً تعريف معالج الحدث، وهو دالة (callback function) تحتوي على الكود الذي نريد تنفيذه عند وقوع الحدث.
2. بعد ذلك نقوم بربط هذه الدالة بعنصر معين وبنوع حدث معين.
3. عندما يقوم المستخدم بالفعل المطلوب، مثل النقر على زر، يقوم المتصفح تلقائياً باستدعاء دالة معالج الحدث وتنفيذ الكود الموجود داخلها.
4. لتسجيل معالجات الأحداث، يُستخدم الأسلوب `addEventListener`، وهو الأسلوب الأحدث والأكثر مرونة.
5. وبما أن معالج الحدث غالباً لا يُستخدم إلا مرة واحدة، فمن الشائع جداً كتابة الدالة مباشرة داخل `addEventListener` على شكل دالة مجهولة. يمكن كتابة هذه الدالة بالطريقة التقليدية، أو باستخدام أسلوب أكثر اختصاراً عبر الدوال السهمية (Arrow Functions)، والتي تعد الصيغة الأحدث والأكثر شيوعاً في JavaScript الحديثة.

مثال:

```
<input type="submit" id="btn">
<script>
  function simpleHandler() {
    alert("button was clicked");
  }
  const btn = document.querySelector("#btn");
  btn.addEventListener("click", simpleHandler);
</script>
```

1 Event handler defined

2 Event handler registered

FIGURE 9.10 JavaScript event handling

Dr. Ghandi Hssam

24

الدالة المجهولة:

الدالة المجهولة (Anonymous Function) هي دالة بدون اسم، نكتبها مباشرة في المكان الذي نحتاجها فيه، وغالباً تُستخدم مع الأحداث لأننا لا نحتاج لإعادة استخدامها في مكان آخر

3 طرق مختلفة لربط حدث (Event) بعنصر في الصفحة

الطريقة الأولى: دالة مجهولة تقليدية

```
const btn = document.getElementById("btn");
btn.addEventListener("click", function () {
  alert("used an anonymous function");
});
```

- أولاً نحدد الزر ونخزنه في متغير btn.
- بعدها نربط حدث click بالزر.
- عند النقر، يتم تنفيذ دالة مجهولة مكتوبة بالطريقة التقليدية. function () {}
- هذه الطريقة واضحة وسهلة القراءة، ومناسبة للمبتدئين.

ملاحظة:

الدالة المجهولة هي:

```
function () { alert("used an anonymous function");}
```

الطريقة الثانية: الأسلوب المتسلسل (Chaining)

```
document.querySelector("#btn").addEventListener("click", function () {
    alert("a different approach but same result");
});
```

- هنا لم نستخدم متغير.
- قمنا بتحديد الزر مباشرة ثم ربط الحدث في سطر واحد.
- النتيجة هي نفسها تماماً مثل الطريقة الأولى.
- هذا الأسلوب أقصر، لكنه قد يكون أقل وضوحاً إذا كان الكود كبيراً.

ملاحظة:

الدالة المجهولة هي نفسها:

```
function () { alert("a different approach but same result");}
```

الاختلاف فقط اننا لم نخزن الزر بمتغير

الطريقة الثالثة: الدالة السهمية (Arrow Function)

```
document.querySelector("#btn").addEventListener("click", () => {
    alert("arrow syntax but same result");
});
```

- نفس الطريقة الثانية، لكن بدلا من الدالة التقليدية استخدمنا الدالة السهمية. () => {}
- هذه الصيغة أحدث وأكثر اختصاراً.
- تستخدم كثيراً في JavaScript الحديثة لأنها أنظف وأسهل كتابة.
- مناسبة جداً لمعالجات الأحداث القصيرة والبسيطة.

ملاحظة:

الدالة المجهولة هنا هي:

```
() => { alert("arrow syntax but same result");}
```

لكن بصيغة Arrow Function

في النهاية، فهم كيفية بناء عناصر الصفحة ديناميكياً والتعامل مع الأحداث هو الأساس لإنشاء صفحات ويب تفاعلية وسلسة، لكن يبقى هناك عامل مهم يجب الانتباه له، وهو توقيت تنفيذ كود الـ DOM بالنسبة لعملية تحميل الصفحة

مفاهيم متقدمة في الأحداث ومعالجة الـ DOM

في هذا الجزء ننتقل من الأساسيات إلى مفاهيم أعمق في التعامل مع الـ DOM والأحداث. الهدف هنا هو فهم متى يجب أن يُنفذ الكود، كيف نعرف تفاصيل الحدث الذي حصل، وكيف ينتقل الحدث بين العناصر داخل الصفحة.

٤ تحميل الصفحة وتنفيذ كود الـ DOM

إحدى المشاكل الشائعة في JavaScript هي محاولة التعامل مع عنصر في الصفحة قبل أن يتم تحميله. في هذه الحالة، سيظهر خطأ لأن العنصر غير موجود بعد في الـ DOM. لحل هذه المشكلة، يجب التأكد من أن كود التعامل مع عناصر الصفحة لا يعمل إلا بعد أن يصبح هيكل HTML جاهزاً. لهذا الغرض، يوفر المتصفح أحداثاً خاصة بتحميل الصفحة.

هناك حدثان مهمان:

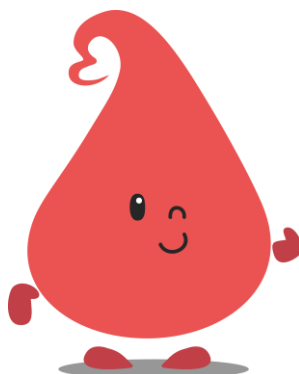
- `window.load`

يتم تشغيله بعد تحميل الصفحة بالكامل، بما في ذلك الصور وملفات CSS والخطوط. هذا الحدث يضمن أن كل شيء جاهز، لكنه بطيء نسبياً لأنه ينتظر جميع الموارد.

- `documentDOMContentLoaded`

يتم تشغيله فور انتهاء المتصفح من تحميل وتحليل ملف HTML فقط، دون انتظار الصور أو الملفات الخارجية. لهذا السبب يُعد الخيار الأفضل والأكثر استخداماً.

أفضل ممارسة هي وضع كل كود التعامل مع الـ DOM داخل مستمع لحدث `DOMContentLoaded`، لضمان أن العناصر موجودة وجاهزة للاستخدام.



```
document.addEventListener('DOMContentLoaded', function() {
    const menu = document.querySelectorAll("#menu li");
    for (let item of menu) {
        item.addEventListener("click", function () {
            item.classList.toggle('shadow');
        });
    }

    const heading = document.querySelector("h3");
    heading.addEventListener('click', function() {
        heading.classList.toggle('shadow');
    });
});
```

LISTING 9.4 Wrapping DOM code within a DOMContentLoaded event handler

Dr. Ghandi Hssam

27

شرح توضيحي:

الكود يعمل شيئين رئيسيين:

1. ينتظر تحميل الصفحة (HTML) بالكامل

2. يضيف تفاعل (click) على:

○ عناصر القائمة داخل #menu

○ عنوان <h3>

وعند النقر على أي واحد منهم، يتم تبديل (toggle) كلاس CSS اسمه shadow.

أولاً: انتظار تحميل الصفحة

document.addEventListener('DOMContentLoaded', function () {

"لا تنفذ أي شيء داخل هذه الدالة إلا بعد أن يتم تحميل وتحليل الـ HTML بالكامل"

وهو المهم

• لأن querySelector و querySelectorAll يفشلون إذا لم تكن العناصر موجودة بعد.

• هي أفضل ممارسة بدل window.onload.

ثانياً: التعامل مع عناصر القائمة

const menu = document.querySelectorAll("#menu li");

هنا: نحدد كل عناصر الموجودة داخل عنصر id اسمه menu و النتيجة NodeList: قائمة عناصر

المرور على عناصر القائمة:

{ for (let item of menu) <= حلقة تمر على كل واحد واحد

إضافة حدث click لكل عنصر:

```
item.addEventListener("click", function () {
    item.classList.toggle('shadow');
});
```

عند النقر على أي:

```
classList.toggle('shadow')
```

○ إذا الكلاس موجود → يتم حذفه

○ إذا غير موجود → يتم إضافته

يعني:

نقرة = تفعيل ظل

نقرة ثانية = إزالة الظل

ثالثًا: التعامل مع العنوان <h3>

```
const heading = document.querySelector("h3");
```

نحدد أول عنصر <h3> في الصفحة

إضافة حدث click للعنوان:

```
heading.addEventListener('click', function () {
    heading.classList.toggle('shadow');
});
```

نفس الفكرة تمامًا:

○ عند النقر على <h3>

○ يتم إضافة أو إزالة كلاس shadow

كائن الحدث (Event Object)

عندما يحدث أي تفاعل (مثل النقر أو الضغط على زر)، يقوم المتصفح بإنشاء كائن حدث يحتوي على معلومات تفصيلية حول ما حصل.

يمكن الوصول إلى هذا الكائن عن طريق تمريره كوسيط إلى دالة معالج الحدث، وغالبًا يُسمَّى e أو event.

من أهم خصائص هذا الكائن:

• e.target

تشير دائمًا إلى العنصر الفعلي الذي قام المستخدم بالتفاعل معه، حتى لو كان الحدث مسجلًا على عنصر أب.

هذه الخاصية مهمة جدًا عندما نستخدم معالجًا واحدًا للتعامل مع عدة عناصر، لأنها تسمح لنا بمعرفة أي عنصر تم النقر عليه بالضبط.

على الرغم من إمكانية استخدام `this` أو المتغيرات الخارجية، إلا أن `e.target` هو الخيار الأكثر مرونة وموثوقية، خاصة في الحالات المتقدمة.

❏ امثلة:

أولاً: الطريقة الأساسية – باستخدام `event object`

```
const menu = document.querySelectorAll("#menu li");
for (let item of menu) {
  item.addEventListener("click", menuHandler);
  function menuHandler(e) {
    const x = e.clientX;
    const y = e.clientY;
    displayArrow(x, y);
    e.target.classList.toggle("selected");
    performMenuAction(e.target.innerHTML);
  }
}
```

ما الذي يحدث هنا؟

1. نضيف `listener` لكل `<li>`

2. عند النقر:

○ المتصفح يرسل كائن الحدث تلقائياً `e`

3. `e` يحتوي معلومات عن:

مكان الماوس (`clientX`, `clientY`)

العنصر الذي انضغط فعلاً `e.target`

4. `e.target`

هذا دائماً يشير إلى **العنصر الذي تم النقر عليه فعلاً**

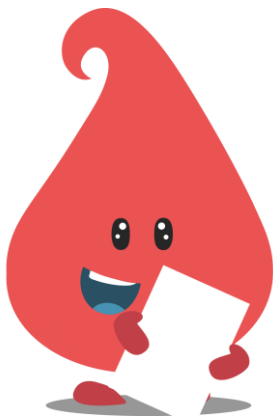
حتى لو كانت الدالة `menuHandler` معرفة في مكان آخر.

لذلك هذه الطريقة **الأكثر أماناً ومرونة**

ثانياً: بدائل بدون `event object`

البديل (1) باستخدام `lexical scope`

```
for (let item of menu) {
  item.addEventListener("click", function () {
    item.classList.toggle("selected");
    performMenuAction(item.innerHTML);
  });
}
```



لماذا يعمل:

- لأن item داخل النطاق (scope) نفسه
- JavaScript ليتذكر قيمة item لكل دورة البديل (2) باستخدام **this**

```
for (let item of menu) {
  item.addEventListener("click", function () {
    this.classList.toggle("selected");
    performMenuAction(this.innerHTML); });
}
```

لماذا يعمل

- داخل function العادية
- this يشير إلى العنصر الذي أطلق الحدث

انتبه: هذا لا يعمل مع arrow function

ثالثاً: أمثلة لا تعمل

المثال الأول (lexical scope) مكسور

```
for (let item of menu) {
  item.addEventListener("click", function () {
    menuHandler();});
}

function menuHandler() {
  item.classList.toggle("selected");
}
```

لماذا لا يعمل

- item غير موجود داخل الدالة
- خرجنا من النطاق

المثال الثاني (this) يشير لشيء خاطئ

```
for (let item of menu) {
  item.addEventListener("click", function () {
    menuHandler();});
  function menuHandler() {
    this.classList.toggle("selected");}
}
```

- this هنا لا يشير للعنصر
- يشير لـ window أو undefined
- لأن menuHandler ليست مربوطة بالحدث مباشرة

انتشار الأحداث (Event Propagation)

عندما ينقر المستخدم على عنصر ما في الصفحة، فإن الحدث لا يبقى محصوراً في هذا العنصر فقط، بل ينتقل عبر العناصر المتداخلة في شجرة الـ DOM. هذه الآلية تسمى انتشار الأحداث. تخيل أن لديك زرّاً داخل div ، والـ div داخل عنصر أكبر. عند النقر على الزر، المتصفح لا يرى الحدث كشيء معزول، بل يجعله يمر بمراحل محددة.

مراحل انتشار الحدث

يمر الحدث بمرحلتين أساسيتين:

1. مرحلة الالتقاط (Capture Phase)

في هذه المرحلة، يبدأ الحدث من أعلى شجرة الـ DOM مثل <html> أو <body> وينتقل نزولاً عبر العناصر الأب حتى يصل إلى العنصر الذي نُقر عليه فعلياً.

2. مرحلة الفقاعات (Bubbling Phase)

بعد وصول الحدث إلى العنصر المستهدف، يبدأ بالتحرك بالعكس: من العنصر الذي نُقر عليه صعوداً إلى العناصر الأب، واحداً تلو الآخر، مثل فقاعة تصعد إلى الأعلى.

بشكل افتراضي، JavaScript تتعامل مع الأحداث في مرحلة الفقاعات.

أين تكمن المشكلة:

أحياناً يكون هذا السلوك غير مرغوب فيه.

مثلاً: في عربة تسوّق، يوجد زر "+" داخل عنصر منتج. عند النقر على الزر:

- يتم تنفيذ كود زيادة الكمية
- لكن أيضاً يتم تنفيذ كود خاص بالعنصر الأب (مثل حذف المنتج أو إغلاق العربة)

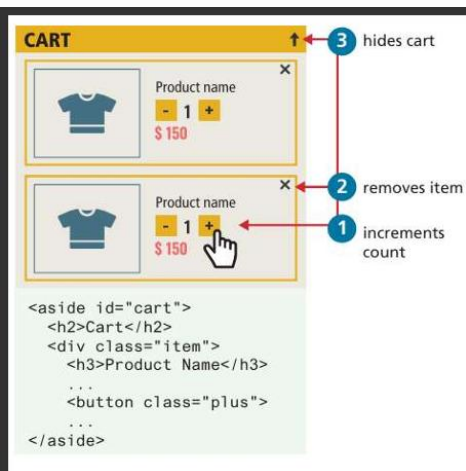
وهذا يحدث لأن الحدث صعد (bubble) إلى العناصر الأصل.

الحدث ينطلق من العنصر نفسه ثم يصعد للأب ثم للأب الأعلى وهكذا...

اي:

في كل عنصر عنده `addEventListener` لنفس نوع الحدث → (click) الكود يتنفذ

يعني ليس مجرد مرور وزيادة تكلفة اداء انما ايضا تنفيذ فعلي للـ logic للأبء التي لها نفس نوع الحدث



الحل: إيقاف انتشار الحدث

لحل هذه المشكلة، نستخدم الدالة: `e.stopPropagation()`

هذه الدالة تطلب من المتصفح: **"توقّف هنا، لا تكمل صعود الحدث للعناصر الأب."**

مثال:

```
const btns = document.querySelectorAll(".plus");
for (let b of btns) {
  b.addEventListener("click", function (e) {
    e.stopPropagation(); // إيقاف انتشار الحدث
    incrementCount(e); });
}
```

بهذا الشكل:

- زر "+" ينفذ عمله فقط
- ولا يؤثر على العناصر التي تحتويه

🔗 تفويض الأحداث (Event Delegation)

الآن نأتي لفكرة ذكية جداً، تعتمد على نفس سلوك الفقاعات، لكن بدل أن نمنعه... نستخدمه لصالحنا. المشكلة: لنفترض أن لدينا قسمًا يحتوي على عشرات أو مئات الصور المصغرة. الطريقة المباشرة هي إضافة مستمع حدث لكل صورة:

```
const images = document.querySelectorAll("#list img");
for (let img of images) {
  img.addEventListener("click", someHandler);
}
```

هذه الطريقة:

- تستهلك ذاكرة كبيرة
- غير عملية إذا زاد عدد العناصر
- لا تعمل مع العناصر التي تُضاف لاحقًا ديناميكيًا

الحل الذكي: تفويض الأحداث

بدل إضافة مستمع لكل عنصر فرعي، نضيف **مستمعًا واحدًا فقط للعنصر الأب.**

الفكرة:

- الحدث سيمرّ تلقائيًا (bubbling)
- نلتقطه في العنصر الأب

- نتحقق: من هو العنصر الذي نُقر عليه فعليًا

المثال:

```
const parent = document.querySelector("#list");
parent.addEventListener("click", function (e) {
  if (e.target && e.target.nodeName == "IMG") {
    doSomething(e.target);
  }
});
```

ماذا يحدث هنا

- النقر يتم على صورة
- الحدث يصعد إلى <section id="list">
- داخل المعالج:
 - نستخدم e.target لمعرفة العنصر الحقيقي الذي أطلق الحدث
 - إذا كان صورة → ننفذ الإجراء

لماذا تفويض الأحداث مهم

- أكثر كفاءة في الذاكرة
- كود أنظف وأسهل صيانة
- يعمل مع العناصر المضافة مستقبلاً بشكل ديناميكي
- يعتمد على سلوك طبيعي في DOM (bubbling)

أي:

- Event Propagation يشرح كيف يتحرك الحدث داخل DOM
- Bubbling هو السلوك الافتراضي
- stopPropagation() نستخدمه لمنع مشاكل غير مرغوبة
- Event Delegation تقنية احترافية تستغل bubbling بدل مقاومته



انتهت المحاضرة

<The end>